

Métodos de Ensamble: *Boosting*, AdaBoost, Gradient Boosting y XGBoost

Tutorial 13 – Aprender secuencialmente de los errores

Luciana Azul Ramirez

E337 – Big Data para Economistas
Universidad de San Andrés

Otoño 2026

Primera parte: las herramientas del día

- 1 **¿Por qué boosting?** La otra familia de ensambles (vs. *bagging*/RF de Tut. 12).
- 2 **AdaBoost:** re-pesar observaciones mal predichas.
- 3 **Gradient Boosting:** cada árbol predice los *residuos* del anterior.
- 4 **XGBoost:** gradient boosting con regularización explícita y escala.
- 5 Tabla comparativa e hiperparámetros en `sklearn/xgboost`.

Segunda parte: papers aplicados

- 1 **Chen & Guestrin (2016, KDD)** – *XGBoost: A Scalable Tree Boosting System*.
- 2 **Mullainathan & Spiess (2017, JEP)** – *Machine Learning: An Applied Econometric Approach*.

Tercera parte: notebook + bonus + supervivencia

Tut13_Ensamble2.ipynb (`scikit-learn/xgboost`), Tut13_ConditionalTrees_Bonus.ipynb (*ctree* en Python) y una *intro* a análisis de supervivencia.

Parte I

De promediar a *corregir secuencialmente*: idea, algoritmo, hiperparámetros

¿Por qué boosting?

- En el Tutorial 12 se vio *bagging* y *random forests*: entrenar muchos árboles **en paralelo** sobre muestras bootstrap y *promediar* sus predicciones. Sirve para **reducir varianza**.
- *Boosting* es la otra gran familia de ensambles. La idea es distinta: los árboles se entrenan **secuencialmente**, y cada uno trata de **corregir los errores del anterior**.
- Combinar muchos *weak learners* (típicamente árboles *chiquitos*) genera un **strong learner**. Esto reduce principalmente el **sesgo**, no la varianza.
- Riesgo principal: **overfitting** si se deja al algoritmo correr demasiado. Se controla con hiperparámetros (`learning_rate`, `n_estimators`, profundidad).

Idea clave

En *bagging* los árboles se entrenan **independientemente y se promedian**. En *boosting* se entrenan **en secuencia, cada uno aprendiendo de los errores del anterior**.

Bagging vs. Boosting

	Bagging / RF	Boosting
¿Cómo se entrenan los árboles?	En paralelo, independientes	En secuencia, cada uno depende del anterior
¿De qué muestra?	Muestras bootstrap distintas	Misma muestra, con <i>pesos</i> o <i>residuos</i> actualizados
Tipo de árbol base	Grandes, sin podar	Pequeños (típicamente <code>max_depth 1-6</code>)
Reduce sobre todo	Varianza	Sesgo
Sensibilidad a hiperparámetros	Baja (defaults andan)	Alta (hay que calibrar)
Riesgo principal	Computacional	Overfitting si no se controla

- Las dos familias son complementarias: cubren los dos lados del trade-off sesgo-varianza.
- En la práctica, *boosting* suele ganar en performance, pero requiere más cuidado en la calibración.

La idea en una oración: se entrenan árboles secuenciales, y en cada paso se les da *más peso* a las observaciones que el modelo anterior predijo mal.

Algoritmo en tres pasos

- ❶ Empezar con pesos iguales para todas las observaciones: $w_i = 1/n$.
 - ❷ Para $b = 1, \dots, B$:
 - Entrenar un árbol *chiquito* (*stump* con `max_depth=3` por defecto en `sklearn`) con los pesos actuales.
 - Calcular el error ponderado del árbol y aumentar el peso de las observaciones mal predichas.
 - ❸ La predicción final es un **promedio ponderado** de los B árboles (los mejores pesan más).
-
- Hiperparámetros: `n_estimators` (B) y `learning_rate` (λ , controla cuánto contribuye cada árbol).

Cambio respecto de AdaBoost: en vez de *re-pesar observaciones*, cada árbol nuevo se entrena para predecir los **residuos** del modelo acumulado hasta ahora.

El algoritmo, intuitivamente

- ❶ Empezar con una predicción inicial $F_0(x)$ (en regresión suele ser el promedio $\bar{y}$).
- ❷ Para $b = 1, \dots, B$:
 - **Calcular el error que falta:** $r_i = y_i - F_{b-1}(x_i)$ (diferencia entre lo verdadero y lo predicho hasta ahora).
 - **Entrenar un árbol** $\hat{f}^{(b)}$ que aprenda a predecir ese error a partir de x_i .
 - **Actualizar el modelo:** $F_b(x) = F_{b-1}(x) + \eta \hat{f}^{(b)}(x)$ (modelo anterior + una fracción η del nuevo árbol).
- **Predicción final:** $\hat{y} = F_0(x) + \eta \hat{f}^{(1)}(x) + \dots + \eta \hat{f}^{(B)}(x)$. Es la suma del punto de partida más todas las correcciones acumuladas.
- “Gradiente” viene de que ajustar a los residuos equivale a dar un paso en el **gradiente negativo** de la pérdida cuadrática $L(y, F) = \frac{1}{2}(y - F)^2$ (la función que el algoritmo trata de minimizar).
- Hiperparámetros: η (learning_rate), B (n_estimators), max_depth del árbol base.

El trade-off η vs. B

η chico, muchos árboles

¿Por qué?

El *learning rate* η controla cuánto contribuye cada árbol nuevo. Cuando η es chico, el modelo aprende *despacio pero estable*; cuando es grande, aprende rápido pero puede sobreajustar y oscilar.

- **Regla práctica:** a *menor* η se necesita *mayor* B para compensar (y viceversa).
- Defaults razonables: $\eta \in [0,01, 0,1]$ con $B \in [100, 1000]$.
- Esa es la **mecánica de regularización** del boosting: muchos pasos chicos generalizan mejor que pocos pasos grandes.
- En sklearn la combinación (η, B) se calibra junto con `max_depth` por *cross-validation* (`GridSearchCV`). En el notebook se hace este procedimiento.

XGBoost = Gradient Boosting + regularización explícita + ingeniería de software. Es hoy el *default* en competencias Kaggle y en muchas aplicaciones industriales.

Función objetivo

$$\text{Obj}(\Theta) = \underbrace{\sum_{i=1}^n L(y_i, \hat{y}_i)}_{\text{pérdida}} + \underbrace{\sum_{m=1}^M \Omega(f_m)}_{\text{regularización}}, \quad \Omega(f) = \gamma T + \frac{\lambda}{2} \sum_{j=1}^T w_j^2.$$

Interpretación: se minimiza el error de predicción (primer término) **más** un castigo por complejidad de cada árbol (segundo término). T : número de hojas; w_j : peso de cada hoja; γ, λ : hiperparámetros que regulan cuánto se castiga *tener muchas hojas* y *tener pesos grandes*.

- Penaliza *explícitamente* la complejidad de cada árbol (cantidad de hojas + magnitud de los pesos). Eso lo hace más robusto a overfitting que GB clásico.
- Maneja **missing values** de forma nativa (*sparsity-aware split*).
- Paralelización y uso de caché lo hacen **mucho más rápido** que `sklearn.GradientBoostingRegressor` en datasets grandes.

AdaBoost vs. Gradient Boosting vs. XGBoost

	AdaBoost	Gradient Boosting	XGBoost
¿Qué actualiza?	Pesos de las observaciones	Residuos (gradiente de la pérdida)	Residuos + regularización del árbol
Función de pérdida	Exponencial (fija)	Cualquier diferenciable	Cualquier diferenciable
Regularización	Implícita (<code>learning_rate</code>)	Implícita (<code>learning_rate</code> , <code>max_depth</code>)	Explícita (γ , λ)
Missing values	No nativo	No nativo	Nativo
Velocidad	Lenta	Lenta-media	Rápida
Año	1997	2001	2016

- *Gradient Boosting* generaliza *AdaBoost*: AdaBoost es un caso particular con pérdida exponencial.
- *XGBoost* es la implementación moderna de gradient boosting con regularización + ingeniería; suele dar el mejor MSE en la práctica.

Ventajas y desventajas de boosting

Ventajas

- **Performance predictiva** top: suele ganar a RF en datasets medianos–grandes.
- Reduce sesgo y varianza al mismo tiempo (con η chico).
- Captura interacciones y no linealidades, igual que CART/RF.
- XGBoost maneja *missing values* nativamente.
- Feature importance: igual que en RF.

Desventajas

- Hiperparámetros **sensibles**: hay que calibrar η , B y profundidad con cuidado.
- **Riesgo de overfitting** si se deja correr demasiado.
- Computacionalmente más caro que RF (es secuencial, no paralelizable a nivel de árbol).
- Caja negra: solo *importance*, no coeficientes.
- Menos robusto a outliers que RF (sobre todo AdaBoost).

- Regla práctica: **Random Forest** como *baseline* (default razonable) y **XGBoost** cuando se busca máxima performance y vale la pena calibrar.

Hiperparámetros en sklearn / xgboost

Argumento	Qué controla	Comentario
n_estimators	Número de árboles secuenciales B	Default 50 (AdaBoost) o 100 (GB/XGB). Calibrar junto con learning_rate.
learning_rate (eta)	Cuánto contribuye cada árbol	El hiperparámetro central. Defaults: 1 (AdaBoost), 0,1 (GB), 0,3 (XGB).
max_depth	Profundidad del árbol base	Chico a propósito (3–6). Árboles grandes pierden la lógica de boosting.
gamma, lambda	Regularización del árbol (solo XGB)	Penalizan complejidad y magnitud de los pesos de hoja.
subsample	Fracción de obs. por árbol	<1 introduce aleatoriedad (<i>stochastic boosting</i>); ayuda contra overfitting.

- Clases en `sklearn.ensemble`: `AdaBoostRegressor`, `GradientBoostingRegressor` (y sus `Classifier`). XGBoost: `xgboost.XGBRegressor`, `XGBClassifier`.
- Boosting **no** requiere estandarizar X (igual que árboles).

Parte II

Dos usos de *boosting* en la frontera de ML y economía

Primer Trabajo

Tianqi Chen & Carlos Guestrin (2016)

XGBoost: A Scalable Tree Boosting System

Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD),
785–794.

- Hacia 2015, gradient boosting ya era el *estado del arte* en problemas predictivos, pero las implementaciones existentes eran **lentas** y no escalaban a datasets de millones de observaciones.
- **Pregunta del paper:** ¿se puede armar un sistema de gradient boosting que sea **rápido**, **escalable**, y a la vez *robusto contra overfitting*?
- **Por qué importa:** en problemas de la industria (publicidad online, detección de fraude, recomendación), las features tienen millones de observaciones y muchas son *sparse* (con missing values).

Aporte del paper

XGBoost es el sistema. Combina (1) una función objetivo **regularizada** (Gradient Boosting + penalización del tamaño del árbol), (2) un algoritmo de *sparsity-aware split* para manejar missing values nativamente, y (3) optimizaciones de sistema (paralelización, uso de caché, compresión).

Las tres innovaciones de XGBoost

1. Función objetivo regularizada

Gradient Boosting clásico minimiza solo la pérdida. XGBoost suma una penalización por complejidad de cada árbol: $\Omega(f) = \gamma T + \frac{\lambda}{2} \sum w_j^2$. Eso lo vuelve más **robusto a overfitting** sin necesidad de *early stopping*.

2. Sparsity-aware split finding

Para datasets con missing values o features sparse (muchas ceros), XGBoost aprende automáticamente una **dirección por defecto** para cada split. No requiere imputación previa.

3. Ingeniería de sistema

Paralelización del cálculo de splits, uso eficiente de caché, compresión de bloques de datos. Resultado: **10× más rápido** que implementaciones previas en datasets grandes.

Validación empírica

- Los autores benchmarkean XGBoost contra `scikit-learn`, R `gbm` y otras implementaciones, en datasets de hasta cientos de millones de filas. XGBoost gana en **velocidad y memoria** sin sacrificar precisión.
- En Kaggle (la plataforma de competencias predictivas), XGBoost fue **el algoritmo ganador** en 17 de 29 competencias estudiadas por los autores en 2015. Casi todas las soluciones top usaban XGBoost o lo combinaban con redes neuronales.

Mensaje

- Para problemas *predictivos puros* a escala industrial, **XGBoost es el primer modelo a probar**: combina el poder estadístico del gradient boosting con una ingeniería que lo hace usable en producción.
- Lección metodológica: a veces el avance no está en un *algoritmo nuevo* sino en una *ingeniería superior* sobre algoritmos conocidos.

Segundo Trabajo

Sendhil Mullainathan & Jann Spiess (2017)

Machine Learning: An Applied Econometric Approach

Journal of Economic Perspectives, 31(2), 87–106.

- ML llegó a la economía aplicada, pero el lenguaje y las prioridades son distintos a las de la econometría tradicional. Hay confusión: ¿cuándo conviene ML, cuándo OLS?
- **Pregunta del paper:** ¿en qué tipo de problemas de economía aplicada *ayuda* ML, y cómo encaja con la pregunta de **predicción vs. estimación causal**?
- Distinción central que introducen: el economista clásico estima $\hat{\beta}$ (un parámetro estructural, p.ej. una elasticidad). ML estima $\hat{y}$ (una predicción). Son tareas distintas y los criterios de éxito también.

Mensaje del paper

Muchas preguntas de política pública son en realidad **problemas de predicción**, no de estimación. Para esas, ML (incluyendo boosting) es la herramienta adecuada y supera a OLS.

Aplicación: predicción de precios de viviendas

- Los autores comparan **OLS**, **LASSO**, **Random Forest** y **Boosted Trees** en una tarea estándar: predecir el precio de viviendas a partir de sus características (datos de AHS, EE.UU.).
- Métrica: R^2 *out-of-sample*.
- Resultado: los métodos basados en árboles (RF y boosting) **baten a OLS por márgenes grandes** (R^2 test $\approx 0,45$ vs. $\approx 0,30$).
- LASSO mejora marginalmente a OLS; el salto grande es de *lineal* a *no lineal con interacciones automáticas*.

Lectura

El problema no es “lineal vs. no lineal” sino *¿la pregunta es predictiva?* Cuando sí, ML domina porque la relación verdadera tiene interacciones que OLS no puede recuperar sin especificación previa.

Tres lecciones para economistas

- ➊ **Predicción es una tarea legítima** en economía aplicada (*prediction policy problems*: a quién auditar, a quién dar crédito, quién va a abandonar la escuela).
 - ➋ Para esa tarea, **ML (RF, boosting, XGBoost) es la herramienta apropiada**, no OLS. La pérdida de interpretabilidad se compensa con *feature importance* y con la mejora predictiva.
 - ➌ Para *estimación causal*, ML por sí solo **no** es la solución: hay que combinarlo con identificación causal (causal forests, doble ML, etc.).
- El paper *justifica* por qué un economista aplicado debe conocer boosting: no es solo una moda, resuelve problemas de política pública mejor que los métodos clásicos cuando la pregunta es predictiva.

Parte III

*Bonus: Conditional Inference Trees en economía aplicada
+ Intro al análisis de supervivencia*

Bonus – Pregunta y datos del paper aplicado

Cita: Huo, S., Feng, D., Gill, T. M., & Chen, X. (2024). *Childhood Circumstances and Health of American and Chinese Older Adults: A Machine Learning Evaluation of Inequality of Opportunity in Health*. IZA DP No. 16764.

- **Pregunta:** ¿cuánto de la desigualdad observada en la salud de adultos mayores se explica por **circunstancias de la infancia** (variables fuera del control del individuo)? Esto es la clásica *Inequality of Opportunity* (IOP) de Roemer.
- **Datos:** HRS (EE.UU., $n = 2\,434$) y CHARLS (China, $n = 5\,612$), adultos ≥ 60 años. *Outcome:* salud autoreportada (escala 1–5). *Predictores:* 43 (HRS) y 36 (CHARLS) variables de infancia en 7 dominios (ambiente al nacer, SES familiar, relaciones, etc.).
- **Método estándar previo:** Roemer paramétrico con descomposición de Shapley. Problema: impone forma funcional aditiva y requiere **selección arbitraria** de circunstancias.

Por qué CITrees y no CART

Las circunstancias son una mezcla de variables continuas y categóricas con *muchas categorías* (educación de los padres, región, etc.) — justo el caso donde CART sufre el **sesgo de selección de variables** (se demuestra en la Actividad 1 del notebook bonus). CIT, al elegir la variable por un test de permutación con corrección por cardinalidad, evita ese sesgo.

Procedimiento del paper

- 1 Estiman tres modelos: Roemer paramétrico, **Conditional Inference Tree** (CIT) y **Conditional Inference Forest** (CIF, 200 árboles, $\frac{1}{2}$ muestra por árbol, sorteo de variables por split, hiperparámetros por OOB).
 - 2 Cada modelo define grupos homogéneos según circunstancias y predice la salud media del grupo.
 - 3 Calculan IOP como la *fracción del Gini* explicada por las circunstancias.
 - 4 Comparan *out-of-sample MSE* con split 2/3 train – 1/3 test.
- **Variables clave** (vía importancia): salud en la infancia, situación financiera familiar, diferencias regionales.
 - En la próxima slide: resultados cuantitativos y mensaje metodológico.

Resultados (IOP como % del Gini total)

- **EE.UU.:** Roemer estima 35,1 %, CIT 23,0 %, CIF 28,3 %.
- **China:** Roemer 22,7 %, CIT 12,8 %, CIF 20,4 %.
- El método paramétrico (Roemer) **sobreestima** sistemáticamente la *Inequality of Opportunity*.
- **CIF gana en MSE test:** predice mejor IOP fuera de muestra que CIT y que Roemer.

Mensaje metodológico

Uso *sustantivo* de CITrees: no como predictor de un *outcome* individual, sino como **herramienta de descomposición de la varianza** para una pregunta normativa. ML aplicado a equidad de oportunidades.

Outcome *tiempo-hasta-evento*: un tipo distinto de variable

- Hasta ahora se vieron dos tipos de *outcome*:
 - **Continuo** (regresión): precio de vivienda, ingreso, MSE.
 - **Categorico** (clasificación): default sí/no, voto al partido X, accuracy.
- Hay un tercer tipo: **tiempo hasta que ocurre un evento**. Ejemplos típicos en economía:
 - Duración del desempleo (¿cuánto tiempo hasta encontrar trabajo?).
 - Default de crédito (¿cuántos meses hasta caer en mora?).
 - Supervivencia de empresas (¿cuántos años hasta cerrar?).
 - *Churn* de clientes (¿cuándo se da de baja un usuario?).
- Estos problemas son generalizados en economía aplicada, finanzas y demografía.

¿Por qué no podemos usar OLS o clasificación directamente?

Porque aparece un problema específico que no surge en los otros tipos de *outcome*: la **censura**.

El problema de la censura

- **Censura:** para algunos individuos *no se observa el evento* dentro del período de estudio. Solo se sabe que “hasta el momento t^* no había pasado”.
- Ejemplo: estudiar cuánto tarda un desempleado en encontrar trabajo. Algunos lo encuentran (el evento se observa), pero otros siguen desempleados cuando termina el estudio (*censura a derecha*).
- Si simplemente se **ignoran los censurados**, se pierde información y se *subestima* la duración. Si se les imputa el tiempo máximo, también se sesga. OLS no maneja esto bien.

La herramienta central: la función de supervivencia

$$S(t) = P(T > t)$$

Probabilidad de que el evento *no* haya ocurrido todavía al tiempo t . Si el outcome es “cuánto tarda en cerrar la empresa”, $S(5)$ es la probabilidad de seguir abierta a los 5 años.

Estimar $S(t)$

Kaplan-Meier (no paramétrico)

Estimador no paramétrico de $S(t)$ que **maneja correctamente la censura**. Produce la clásica “curva escalonada”. No usa covariables: solo describe el tiempo agregado.

Cox proportional hazards (semiparamétrico)

Modelo de regresión para tiempo-hasta-evento. Modela la **tasa de hazard** (probabilidad instantánea de evento condicional a haber sobrevivido):

$$h(t | x) = h_0(t) \exp(x^\top \beta).$$

$h_0(t)$ queda *libre* (no paramétrico); $\exp(\beta_j)$ se interpreta como un *hazard ratio*.

- En Python: paquete `lifelines` (KM, Cox, hazard ratios). En R: `survival`.

Notebook principal: Tut13_Ensamble2.ipynb

Datos:

- Boston (de ISLP): regresión sobre el valor mediano de viviendas.

Herramientas a utilizar:

- `sklearn.ensemble.AdaBoostRegressor`, `GradientBoostingRegressor`.
- `xgboost.XGBRegressor` + `plot_importance`.
- `sklearn.model_selection.GridSearchCV` para elegir `n_estimators`, `learning_rate`, `max_depth`.
- Comparación final de MSE: MCO, CART, Bagging, RF, AdaBoost, GB, XGBoost.

Notebook *bonus* (opcional): Tut13_ConditionalTrees_Bonus.ipynb

Conditional Inference Trees en Python con el paquete `citrees`. Demostración del *sesgo de selección de variables* de CART y aplicación al paper presentado en la Parte III.

- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1), 119–139.
- Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.
- Hothorn, T., Hornik, K., & Zeileis, A. (2006). Unbiased recursive partitioning: A conditional inference framework. *Journal of Computational and Graphical Statistics*, 15(3), 651–674.
- Huo, S., Feng, D., Gill, T. M., & Chen, X. (2024). Childhood Circumstances and Health of American and Chinese Older Adults: A Machine Learning Evaluation of Inequality of Opportunity in Health. *IZA Discussion Paper No. 16764*.
- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An introduction to statistical learning: with applications in Python*. Cap. 8.2 y 11. Springer.
- Mullainathan, S., & Spiess, J. (2017). Machine Learning: An Applied Econometric Approach. *Journal of Economic Perspectives*, 31(2), 87–106.
- Sosa Escudero, W. (2021). *Big Data*, 7.^a edición, Siglo XXI, Cap. 3.